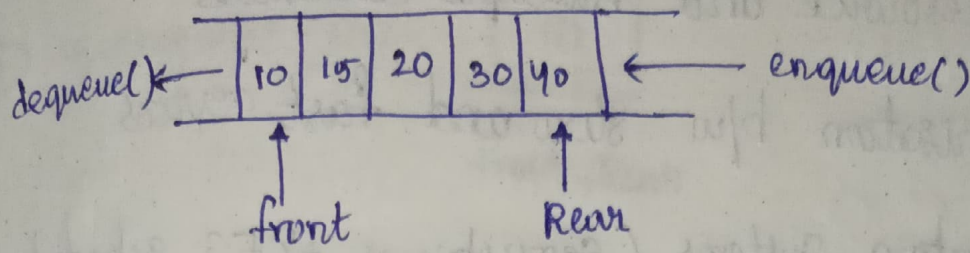


Queue Data Structure

FIFO



operations:

enqueue(x)

dequeue()

getFront()

getRear()

size()

isEmpty()

Queue q

q.enqueue(10) $\Rightarrow$ [10]

q.enqueue(20) $\Rightarrow$ [10 | 20]

q.enqueue(30) $\Rightarrow$ [10 | 20 | 30]

q.enqueue(40) $\Rightarrow$ [10 | 20 | 30 | 40]

print(q.dequeue()) $\Rightarrow$ [20 | 30 | 40]

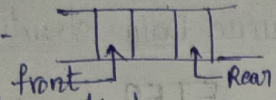
print(q.dequeue()) $\Rightarrow$ [30 | 40]

print(q.getFront()) =

print(q.getRear())

O/p: 10
20
30
40

Applications of Queue:-

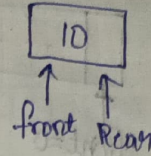


- ↳ * Single resource and multiple consumers
- ↳ * Synchronisation b/w slow and fast devices
- ↳ In operating systems (semaphores, FCFS scheduling, spooling, buffers for devices like keyboards)
- ↳ In computer networks (routers/switches and mail queues)
- ↳ Variation: Deque, priority queue and doubly ended priority queue

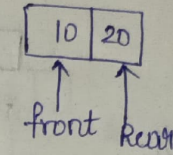
process used in printers

Implementation of queue using Array

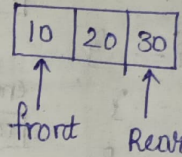
① q.enqueue(10)



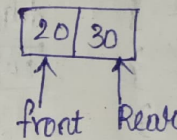
② q.enqueue(20)



③ q.enqueue(30)



④ q.dequeue()



Additional operations:

size()

getFront()

getRear()

isFull()

isEmpty()

Simple Implementation:

struct queue {

int size, cap;

int *arr;

queue(int c)

{ cap = c;

size = 0;

arr = new int[cap];

}

void enqueue(int x) {

void dequeue() {

int getFront() {

int getRear() {

bool isFull() {

bool isEmpty() {

int size() {

}

bool isFull()

{ return (size == cap);

}

bool isEmpty()

{ return (size == 0);

}

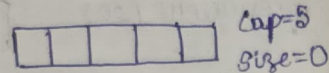
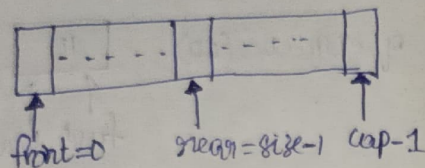
void enqueue(int x)

{ if (isFull()) return;

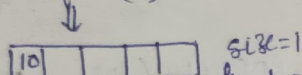
arr[size] = x;

size++;

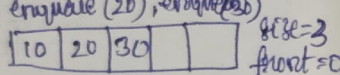
}



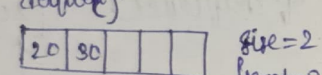
enqueue(10)



enqueue(20), enqueue(30)



dequeue()



void dequeue()

{ if (isEmpty()) return;

for (int i=0; i<size-1; i++)

arr[i] = arr[i+1];

size--;

int getFront()

{ if (isEmpty()) return -1;

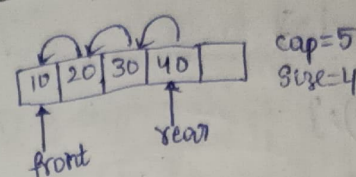
else return 0;

}

int getRear()

{ if (isEmpty()) return -1;

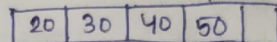
else return size-1;



dequeue()



enqueue(50)



struct Queue

{ int *arr;

int front, cap, size;

Queue(int c)

{ arr = new int[c];

cap = c;

front = 0;

size = 0;

}

/* Function to be added here */

}

dequeue(): front = (front + 1) % cap;

size--;

enqueue(): rear = (rear + 1) % cap;

arr[rear] = x;

size++;

}

dequeue(): front = (front + 1) % cap;

size--;

enqueue(): rear = (rear + 1) % cap;

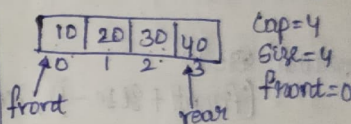
arr[rear] = x;

size++;

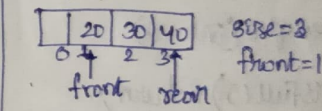
}

To do all operations in O(1)

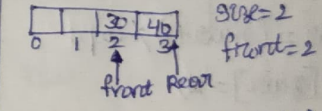
Efficient Implementation (Circular Array)



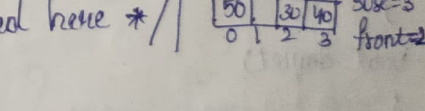
dequeue()



dequeue()



enqueue(50)




```
bool isFull() { return (cap == size); }
```

```
bool isEmpty() { return (size == 0); }
```

```
int getFront()
```

```
{ if (isEmpty())
```

```
    return -1;
```

```
    else return front;
```

```
}
```

```
int getRear()
```

```
{ if (isEmpty())
```

```
    return -1;
```

```
else
```

```
    return size-1
```

```
    (front + size - 1) % cap;
```

```
}
```

```
void enqueue(int x)
```

```
{ if (isFull()) return;
```

```
    int rear = getRear();
```

```
    rear = (rear + 1) % cap;
```

```
    arr[rear] = x;
```

```
    size++;
```

```
}
```

```
void dequeue()
```

```
{ if (isEmpty()) return;
```

```
    front = (front + 1) % cap;
```

```
    size--;
```

```
}
```

cap = 3
size = 0, front = 0

enqueue(10), enqueue(20), enqueue(30)

size = 3, front = 0

dequeue(), dequeue()

size = 1, front = 2

enqueue(40)

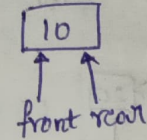
size = 2, front = 2

dequeue()

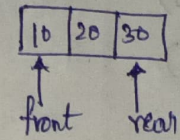
size = 1, front = 0

Linked List Implementation of Queue

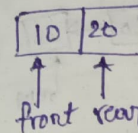
① q.enqueue(10)



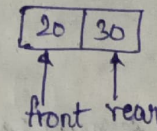
③ q.enqueue(30)



② q.enqueue(20)



④ q.dequeue()



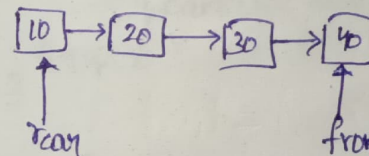
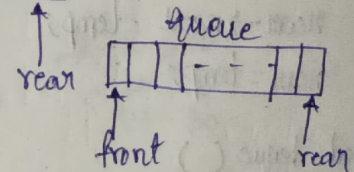
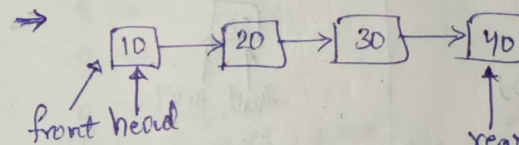
Additional operations

size()

getFront()

getRear()

isEmpty()



wrong choice of front and rear.


```

struct Node
{
    int data;
    Node * next;
    Node (int x)
    {
        data = x;
        next = NULL;
    }
};

```

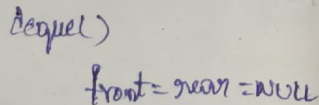
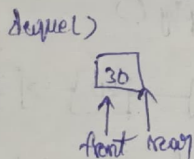
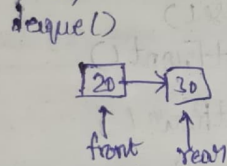
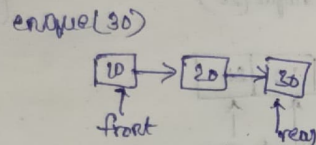
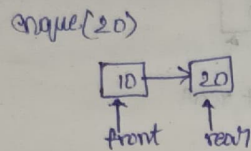
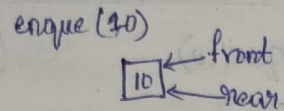
struct Queue

```

{
    Node * front, * rear; int size;
    Queue () { front = NULL; rear = NULL; size = 0; }
    void enqueue (int x)
    {
        Node * temp = new Node (x);
        if (front == NULL)
        {
            front = rear = temp;
            return;
        }
        rear->next = temp;
        rear = temp;
    }
    void dequeue ()
    {
        if (front == NULL) return;
        Node * temp = front;
        front = front->next;
        if (front == NULL) { rear = NULL; }
        delete (temp);
    }
};

```

front = rear = NULL



Queue in C++ STL

```

push()
pop()
front()
back()
empty()
size()

```

```

#include <iostream>
#include <queue>
using namespace std;

```

int main()

```

{
    queue <int> q;

```

```

    q.push(10); [10]

```

```

    q.push(20); [10 20]

```

```

    q.push(30); [10 20 30]

```

```

    cout << q.front() << " " << q.back() << endl;

```

```

    q.pop(); [20 30]

```

```

    cout << q.front() << " " << q.back() << endl;

```

o/p: [10 30]
[20 30]

Queue Traversal:

int main()

```

{
    queue <int> q;

```

```

    q.push(10);

```

```

    q.push(20);

```

```

    q.push(30);

```

```

    cout << q.size() << endl;

```

```

    while (q.empty() == false)
    {

```

```

        cout << q.front() << " "
            << q.back() << endl;

```

```

        q.pop();
    }
}

```

[10 20 30]

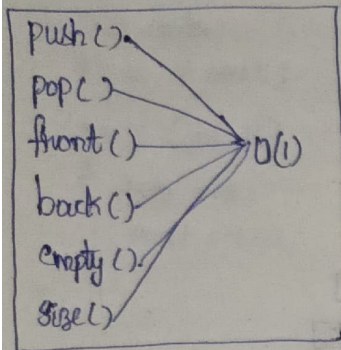
3

o/p: 10 30

20 30

30 30

Time Complexities & Internal Working



Queue can be implemented on any underlying container that provides the following functions

empty()
 size()
 front()
 back()
 push_back()
 pop_front()

list & deque

vector doesn't provide pop_front() in $O(1)$ time.

*By default queue is implemented using deque container
 → Queue is a container adapter, it just works as an interface on top of other containers.

Implement Stack Using Queue

consider a situation where we have library available for queue and we need stack library.
 How to implement the stack.

struct stack

```

{
    int top() { ... }
    void push(int x) { ... }
    int pop() { ... }
    int size() { ... }
}
  
```

struct stack {

queue<int> q1, q2;

int top() { return q1.front(); }

int size() { return q1.size(); }

int pop() { return q1.front(); }

void push(int x)

{ while (q1.empty() == false) {

{ q2.push(q1.top());

q1.pop();

q1.push(x); }

while (q2.empty() == false) {

q1.push(q2.top());

q2.pop(); }

}

q1: [10, 5, 15]

q2: []

↓ 20

q1: []

q2: [10, 5, 15]

q1: [20]

q2: [10, 5, 15]

q1: [20, 10, 5, 15]

q2: []

More Variations:

- 1 Implement stack using Queue by making pop operation costly.
- 2 Implement stack using only one Queue (we use recursion call stack here) $\rightarrow$ reverse of a stack
- 3 Implement Queue using stack
 - $\rightarrow$ By making enqueue() operation costly.
 - $\rightarrow$ By making dequeue() operation costly.
 - $\rightarrow$ By using one stack (and one recursion call stack).

Reversing a Queue

Given a queue, we need to reverse it.

IP: $q = [10, 5, 15, 20]$
front $\uparrow$ rear $\uparrow$

OP: $q = [20, 15, 5, 10]$
front $\uparrow$ rear $\uparrow$

1 Iterative:

void reverse(queue<int> &q)

{ stack<int> s;

while (q.empty() == false)

{ s.push(q.front());

q.pop();

while (s.empty() == false)

{ q.push(s.top());

s.pop();

}

2 Recursive:

void reverse(queue<int> &q) {
if (q.empty())
return;

int x = q.front();

q.pop();

reverse(q);

q.push(x);

}

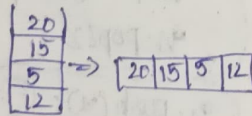
$q = [12, 5, 15, 20]$
 $\uparrow$ front

n & n-1 relation

$q = [5, 15, 20]$

$q = [20, 15, 5]$

$q = [20, 15, 5, 12]$



Generate Numbers with given digits:

Given a number n , print first n numbers (in increasing order) such that all these numbers have digits in set $\{5, 6\}$

examples:

Ip: $n=10$

Op: 5, 6, 55, 56, 65, 66, 555, 556, 565, 566

Ip: $n=4$

Op: 5, 6, 55, 56

NOTE: n can be a big number and the result values might not fit in basic data types like long int or long long int.

void printFirstN(int n)

{

queue<int> q;

q.push("5");

q.push("6");

for(int count=0; count<n; count++)

{ string curr = q.top();

cout << curr << " ";

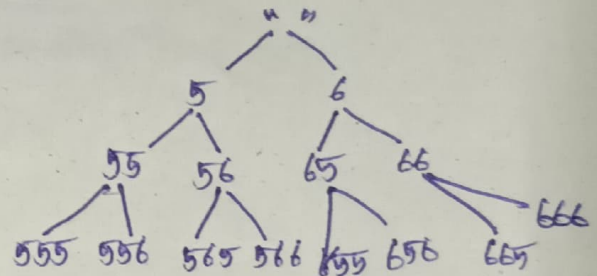
q.pop();

q.push(curr + "5");

q.push(curr + "6");

}

}



q [5 6] $\Rightarrow 5$

q [6 55 56] $\Rightarrow 6$

q [55 56 65 66]

q [55 56 555 556]

T.C = $O(n)$
A.S = $O(n)$